STUDENT CODE: **198E**

Gebze Technical University
Department of Computer Engineering
CSE 241/501
Fall 2024
Midterm Exam Nov 13th 2024

Student Code	Q1 (25)	Q2 (30)	Q3 (45)	Exam Format (-10)
198				

Read the instructions below carefully

- Do not write your name or your student number in any part of the exam materials including the answer sheets.
- Your individual student codes are already printed on your question and answer sheets. Keep this student code until the exam ends. You will use it to sign the exam attendance sheet.
- Answer each question on the designated page in the box only. We will not (and cannot) grade any materials outside of the designated boxes
- Follow the line guides when writing, do not skip lines, do not use more than one line for a single line.
- Hand over your cell phones, books, and notes to the TA before the exam starts. You will not use any electronic devices or books during the exam.
- This exam has 3 questions; it will be graded out of 100 points. You have 120 minutes. If you do not follow any of the rules above, you will lose 10 points.

Question 1

a) What is wrong with the following line? Explain our answer.

```
assert( a++ >= 0)
```

b) What is wrong with the following copy constructor declaration of the class **MyClass**? Explain your answer.

```
MyClass(const MyClass o);
```

c) What are the similarities and differences between global variables and static data members of classes?

Question 2

Implement a global C++ function that takes a 2-dimensional **std::vector** of **Rational** objects as parameter. Your function returns a 1-dimensional **vector** of **Rational** objects, where each element of this **vector** is the median (Turkish ortanca) of the elements of a row. Relevant parts of class **Rational** is given below. You may write additional helper functions. Do not implement any classes. Do not use any library functions for sorting.

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);  // Assume a constructor exists
    bool operator<(const Rational& rhs) const;    // Compares two Rationals
    Rational operator+(const Rational& rhs) const;
    Rational operator/(int value) const;

    // Other necessary member functions and data here
};
```

$\frac{1}{5}$ $\frac{1}{4}$

In case you need, the median is the middle value in a list of numbers sorted in ascending order, or the average of the two middle numbers if the list contains an even number of elements.

STUDENT CODE: **198E**

Question 3

Study the following driver code for a general-purpose reusable class named **GTUStack** whose output is given on the right.

```
#include <iostream>
#include "GTUIntStack.h"
using std::cout;

int main() {
    GTUIntStack stack;
    stack.push(1);
    stack.push(2);
    stack.push(3);
    cout << stack << std::endl;

    GTUIntStack stackCopy(stack);
    stackCopy.push(4);
    cout << stackCopy << std::endl;

    cout << "Size of original stack: "
         << stack.size() << std::endl;
    cout << "Size of copied stack: "
         << stackCopy.size() << std::endl;

    if (!stack.isEmpty())
        cout << "Original stack is not
empty."
             << std::endl;

    stack.pop();
    stack.pop();
    stack.pop();

    if (stack.isEmpty())
        cout << "Original stack is now
empty."
             << std::endl;

    cout << GTUIntStack::getStacksCreated()
         << std::endl;

    return 0;
}
```

Output Screen

```
3 2 1
4 3 2 1
Size of original stack: 3
Size of copied stack: 4
Original stack is not empty.
Original stack is now empty.
2
```

Class **GTUStack** has three data members only: an **int *** to keep the integers, an **int** to keep the capacity of the stack, and another int to show the top of the stack.

Q3 Part a) Write the complete class header. <u>You should not implement any functions in this part</u>.

Q3 Part b) Write the implementation file that includes all the necessary class member and global functions. Do not forget to check for errors.

STUDENT CODE: **198E**

Write your answer for **Question 1** in the box below.

a)

Using post increment/decrement in assert condition is wrong because assert is only used for debugging and when we dont debug the statement wont work and broke the code.

Line also has post increment so even while debugging if the assert statement is false the program will end and post increment wont work.

b)

Sending objects as argument without reference makes the program copy the whole object and it is inefficient. It's okay for small data types to not use reference but bigger data types like class objects requires reference for efficiency.

c)

They are not part of any scope and they are accesible in everywhere in the program. They are created once and used until the end of the program.

STUDENT CODE: **198E**

Write your answer for **Question 2** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
vector<Rational> getMedian(vector<vector<Rational>>& v){

    vector<Rational> medians;

    for(auto i:v){
      int size = i.size(); Rational temp;
      // bubble sort
      for(int j=0; j<size-1; j++){
         for(int k=0; k<size-1-j; k++){
            if(i[j+k+1] < i[j+k]){ // min to max
               temp = i[j+k+1];
               i[j+k+1] = i[j+k];
               i[j+k] = temp;
            }
         }
      }
      if(size%2==1){
        medians.push_back(i[(size/2)+1]);
      }

      else{
    medians.push_back((i[(size/2)+1] + i[(size/2)-1])/2)
    }
  }
  return medians;
}
```

STUDENT CODE: **198E**

Write your answer for **Question 3-a** in the box below.

```
#ifndef GTUINTSTACK_H
#define GTUINTSTACK_H
#include <iostream>  using namespace std;
class GTUIntStack {
public:
    void push(int num);
friend ostream& operator<<(ostream& o, const GTUIntStack& g);
    int size() const;
    bool isEmpty() const;
    void pop();
    GTUIntStack();
    GTUIntStack(const GTUIntStack& cs); //copy constructor
    int operator[](int index)const;


private:
    int* stack;
    int cap;
    int top;

};

#endif
```

STUDENT CODE: **198E**

Write your answer for **Question 3-b** in the box below.

```
#include <iostream>
using namespace std;

GTUIntStack::GTUIntStack(){
    stack=nullptr;
    cap=10;
    top=cap;
}
int operator[](int index)const{if(index>=top){return stack[index];}}

ostream& operator<<(ostream&o, const GTUIntStack& g){
  int i;
  for(i=top;i<cap-1;i++){
    o<< stack[i]<<" ";
  }
  return o;
}

bool GTUIntStack::isEmpty()const{
    return (cap-top==0);
}

int GTUIntStack::size()const{
   return cap-top;
}

GTUIntStack::GTUIntStack(const GTUIntStack& cs){
    int size=cs.size();
    stack=new int[size+10];
    cap=size+10;  top=cap-size;
    for(int i=top;i<cap-1;i++){
       stack[i]=cs[i];
    }
}

void GTUIntStack::push(int num){
    int size=cap-top;
    if(cap<=size+1){cap+=10;}
    int *a=new int[cap];
    for(int i=top;i<cap-1;i++){
      a[i]=stack[i];
    }
    a[top-1]=num;}
    delete[] stack;
    stack=a;
    top--;
}
```

STUDENT CODE: **198E**

Continue your answer for **Question 3-b** in the box below.

```
void GTUIntStack::pop(){
  int *a=new int[cap]; int size=cap-top; if(size>0){
  for(int i=top+1;i<cap-1;i++){
    a[i]=stack[i];
  }
  delete [] stack;
  stack=a;
  top++;
  }
}
```

STUDENT CODE: **198E**

Continue your answer for **Question 3-b** in the box below.